

MyBudget Personal Finance Manager

C322

Group: 14

Presented by

Conner Inman

Maddox Franklin

Caden Batts

Overview

Supports:

- Multiple accounts
- Recurring transactions
- Customizable budget limits
- Real-time budget alerts

Account	Balance	Limit	Balance Health
Checking	\$2720.00	\$5000.00	54% of limit
Savings	\$9750.00	\$10000.00	97% of limit
Vacation Fund	\$70.00	\$2000.00	3% of limit

MyBudget is a desktop personal finance management application designed to help users take control of their budgets, track transactions, and visualize spending patterns

Observer

The observer pattern is used in conjunction with the command pattern. Every time a command is executed, observers are automatically notified so the notification system isn't constantly checking for changes.

Singleton

The singleton pattern is used to ensure only one active user session at a time as well as one app configuration. Both of these being under the singleton pattern guarantees consistent user information.

Command

The command pattern is used for executing deposits, withdrawals, and setting budgets. It also holds a history of previous commands for being able to undo and redo.

Strategy

The strategy pattern is used to separate the varying report formatting behaviors. The app supports 3 different behaviors, PDF, CSV, and a chart.

Model

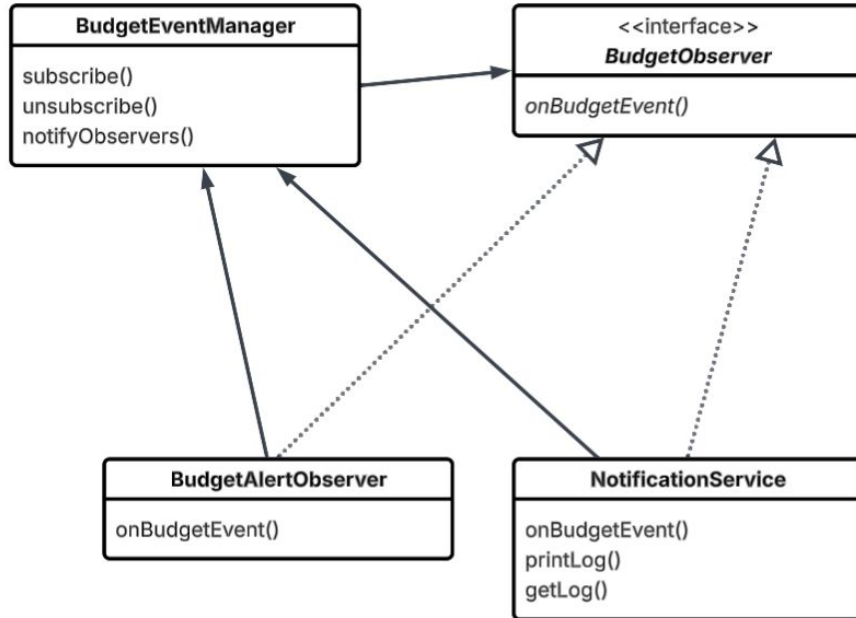
The model is essentially the foundation of our app. It holds the Account and Transaction classes that contain the information used across the app. All the other patterns are built around the model promoting loose coupling.

GUI

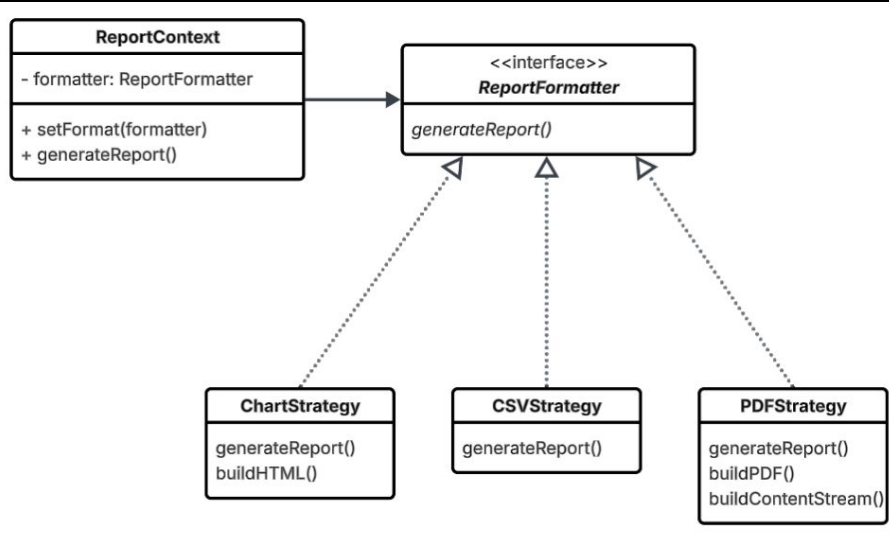
We've also implemented a clean GUI using JavaSwing for a smoother user experience. The GUI unifies the app's features into an easier and more intuitive interface.

Observer Pattern

Used for tracking the budget and spending alert notifications. BudgetEventManager acts as the subject, and everytime a command is executed it notifies the observers, BudgetAlertObserver and NotificationService.

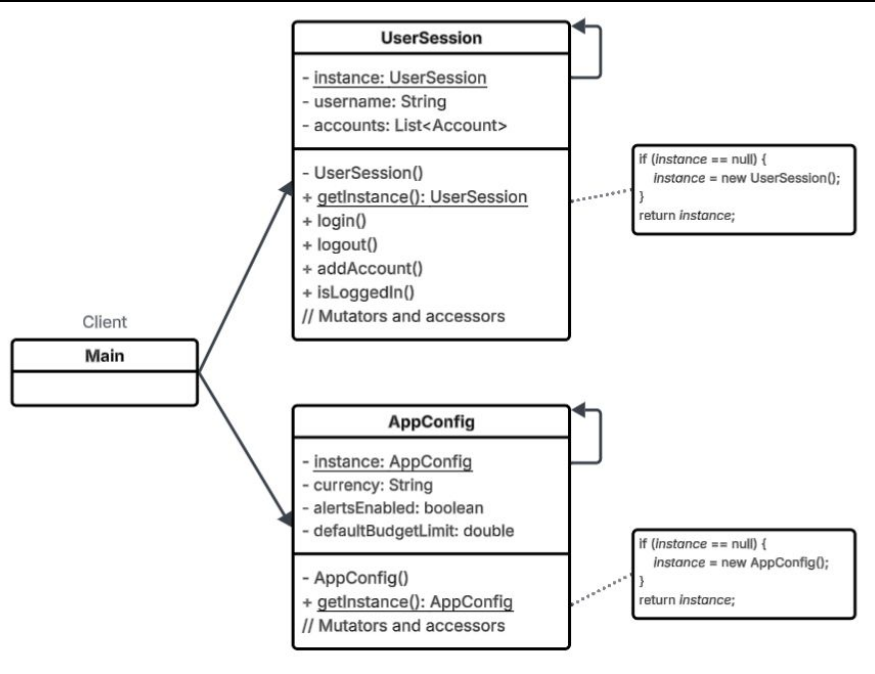


Strategy Pattern



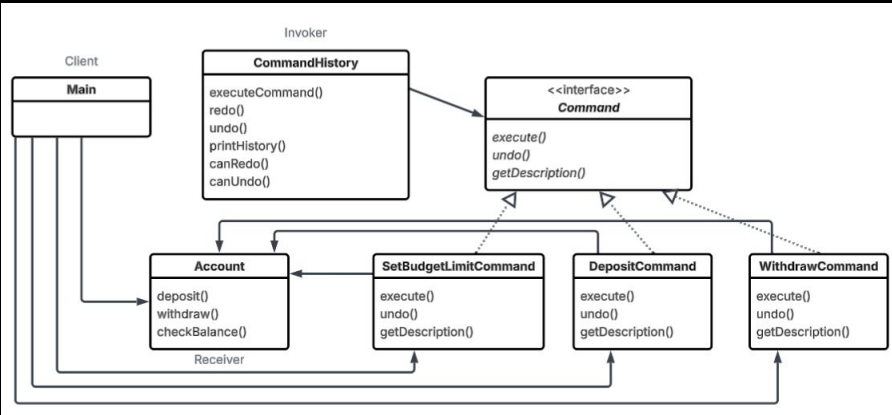
Used for various formatting and file export types. The 3 different report behaviors are separated under the `ReportFormatter` interface and implement their own individual behaviors.

Singleton Pattern



Used to maintain the user's session ensuring only one instance of the application is being used. The 2 singleton instances guarantee consistent data and prevent conflicts.

Command Pattern



Used for executing the functionality of the app. The 3 commands are abstracted away with their own implementations of `execute`, so the invoker can just call `executeCommand` and the corresponding one is used.

Timeline & Contributions

April 14 - 17



Project setup, class diagrams,
Singleton classes, Command base
interface

April 18 - 22



Account model, DepositCommand,
WithdrawCommand,
SetBudgetLimitCommand,
CommandHistory with undo/redo

April 23 - 26



Observer pattern, all three Strategy
implementations, JavaSwing GUI
integration, demo user seeding,
PDF/CSV/Chart output

April 27 - 29



Final report, UML diagrams,
presentation slides, submission

Conner



Overall system architecture, Project
Proposal, develop/debugged code,
Strategy, Command

Maddox



Singleton, Debugger, Presentation,
Code Testing, Command Line
Interface

Caden



Implemented the User Interface: GUI,
Program Demo, Observer alerts

Thank you